🛡️ Repo Shield Dashboard

Loaded report: report.md

Tip: click the ⋮ (three dots) menu in the top-right of the Streamlit app, choose 'Print', then use your browser's 'Save as PDF' option.

🛡️ Repo Shield Security Report

Repository: https://github.com/codemencer-Rahul/Megatronix_Web_2k25
Scan Time: 2026-03-25
Total Findings: 33

📊 Executive Summary

This repository contains **33 potential security issues**, including:

- 🔴 0 Critical Issues (possible secrets / credentials)
- 🟠 1 High Risk Issues (unsafe patterns)
- 🟡 32 Medium Risk Issues (dependency risks)

👉 Immediate attention is required for **credential exposure and secret leakage**.

ℹ️ Note: These counts come from the static scanner/AST layer and represent *suspected* risk before AI review. See the 🤖 AI Audit section for confirmations and false-positive explanations.

📈 Risk Breakdown

Severity (pre-AI)	Count
🔴 Critical	0
🟠 High	1
🟡 Medium	32

🚨 Critical Issues

No critical issues found.

⚠️ High Risk Issues

Unsafe Pattern Usage

- **File:** package-lock.json
- **Reason:** High-entropy secret-like string detected (entropy=5.49)

📦 Dependency Risks (Aggregated)

⚠️ Found **32+ unpinned dependencies**

Example:

- @gsap/react@^2.1.2
- @tailwindcss/vite@^4.1.16
- appwrite@^21.5.0

👉 Risk:

- Unexpected breaking changes
- Supply chain attacks

📋 Detailed Findings

#	File	Severity (pre-AI)	Reason
1	package-lock.json	🔴 High	High entropy string
2	package.json	🟡 Medium	Unpinned dependency

#	File	Severity (pre-AI)	Reason
3	package.json	Medium	Unpinned dependency
4	package.json	Medium	Unpinned dependency
5	package.json	Medium	Unpinned dependency
6	package.json	Medium	Unpinned dependency
7	package.json	Medium	Unpinned dependency
8	package.json	Medium	Unpinned dependency
9	package.json	Medium	Unpinned dependency
10	package.json	Medium	Unpinned dependency
11	package.json	Medium	Unpinned dependency
12	package.json	Medium	Unpinned dependency
13	package.json	Medium	Unpinned dependency
14	package.json	Medium	Unpinned dependency
15	package.json	Medium	Unpinned dependency
16	package.json	Medium	Unpinned dependency
17	package.json	Medium	Unpinned dependency
18	package.json	Medium	Unpinned dependency
19	package.json	Medium	Unpinned dependency
20	package.json	Medium	Unpinned dependency
21	package.json	Medium	Unpinned dependency
22	package.json	Medium	Unpinned dependency
23	package.json	Medium	Unpinned dependency
24	package.json	Medium	Unpinned dependency
25	package.json	Medium	Unpinned dependency
26	package.json	Medium	Unpinned dependency
27	package.json	Medium	Unpinned dependency
28	package.json	Medium	Unpinned dependency
29	package.json	Medium	Unpinned dependency

#	File	Severity (pre-AI)	Reason
30	package.json	🟡 Medium	Unpinned dependency
31	package.json	🟡 Medium	Unpinned dependency
32	package.json	🟡 Medium	Unpinned dependency
33	package.json	🟡 Medium	Unpinned dependency

🤖 AI Audit

⚠️ Note: The following manifest/lock files are *not* sent to AI for auditing and are reported based solely on static scanner/AST analysis. Only manifest/lock files flagged as high risk (secrets / very high-entropy strings) are included in the AI audit.

- package.json

- Files sent to AI audit: **1**
- Successful AI responses: **1**
- Failed AI audits: **0**

#	File	Status	AI Summary
1	package-lock.json	✅ Success	#### 1. Main Risk Summary

🧠 Detailed AI Summaries (per file)

package-lock.json

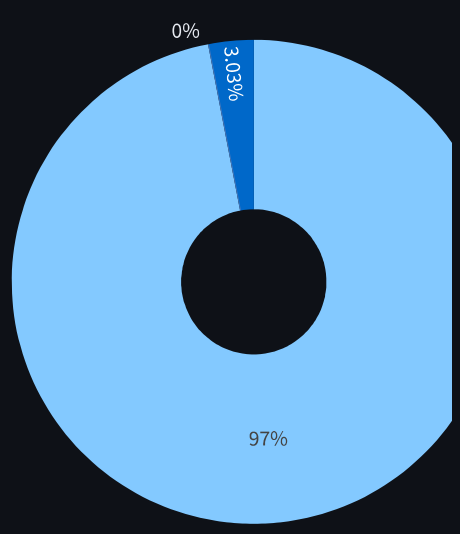
Section	Details
Main Risk	The automated security flag is a **false positive**; high-entropy strings detected in this file are standard cryptographic integrity hashes used by the package manager to verify the authenticity of dependencies, not actual hardcoded secrets or API keys.
Specific Issues	The scanner is flagging the `integrity` fields found throughout the `package-lock.json` file. These strings use Base64 encoding for SHA-512 hashes, which naturally exhibit high entropy (often exceeding 5.0). * **Issue 1: Integrity Hash flagged as Secret** * **Location:** Line 52 (and repeated for every package entry) * **Snippet:** `"integrity": "sha512-cjQ7ZlQ0Mv3b47hABuTevyTuYN4i+loJKGeV9flcCgIK37cCXRh+L1bd3iBHlynerhQ7BhCkn2BPbQUL+rGqFg=="` *

Section	Details
	Context: This is a checksum for the `@babel/code-frame` package to ensure the downloaded code has not been tampered with. * **Issue 2: Integrity Hash flagged as Secret**
Fix Suggestions	Since these strings are required for the project's security and functionality, do not remove them. Instead, follow these steps to manage the alert: * **Mark as False Positive:** Manually resolve the alert in your security scanning tool (e.g., GitHub Advanced Security, TruffleHog, or Gitleaks) as a "False Positive" or "Used in Tests/Metadata." * **Update Scanner Configuration:** Refine your secret scanner's configuration (e.g., `.gitleaksignore` or `exclude` rules) to ignore the `integrity` key within `.json` files or specifically exclude `package-lock.json` from high-entropy checks. * **Verify True Secrets:** Ensure that actual configuration files (like `.env` or `firebase-config.js`) are included in your `.gitignore` to prevent real high-entropy keys (like Firebase or Appwrite API keys) from being committed.

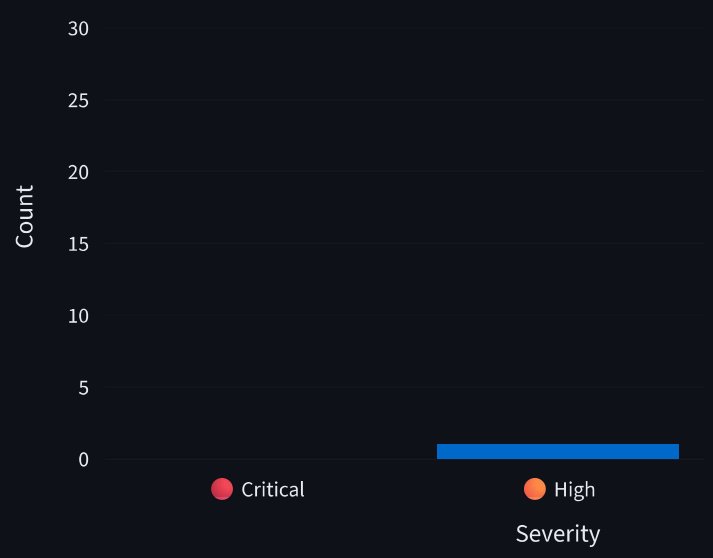
⚙️ Generated by Repo Shield

📊 Charts From Report

Risk Breakdown



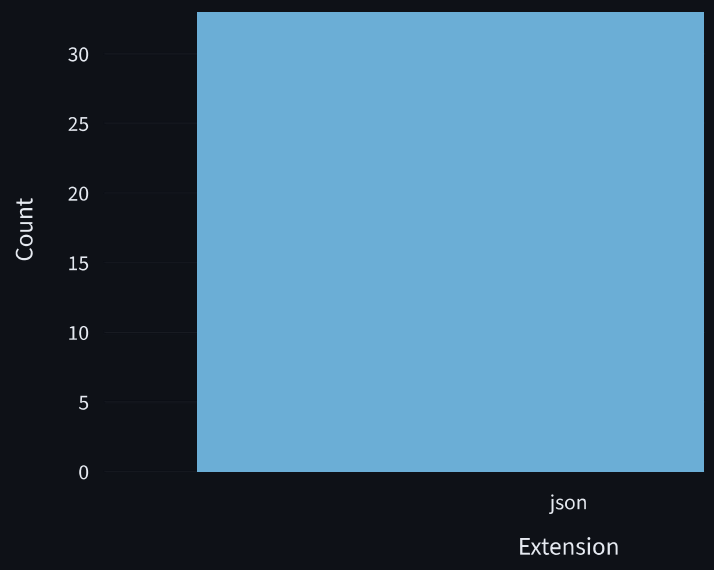
Risk Counts by Severity



Top Finding Reasons

Findings by File Extension

30
25
20
15
10
5
0
Count
Unpinned dependency
Reason


30
25
20
15
10
5
0
Count
json
Extension